SEMINAR

TOPIC:- GRAPH TRAVERSAL
(BREADTH FIRST SEARCH)

TEAM:- 6



CONTENT



CONCLUSION



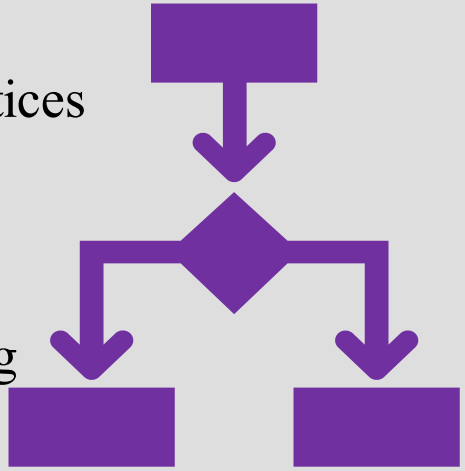
BREADTH FIRST SEARCH(BFS)



INTRODUCTION TO GRAPH TRAVERSAL

INTRODUCTION TO GRAPH TRAVERSAL

A **graph** is a data structure consisting of a set of **vertices (nodes)** and **edges** that connect pairs of vertices.



- A **graph traversal** refers to the process of visiting all the vertices in a systematic way.
- Traversal is necessary for exploring all parts of the graph, searching for specific nodes, checking connectivity, or solving problems like shortest paths, cycle detection, etc.
- While using graph algorithms, we must make sure each vertex is visited once, and the visiting order depends on the algorithm. To avoid revisiting, we usually **mark each vertex** as visited.

There are two primary graph traversal methods:

- **Breadth-First Search (BFS)**
- **Depth-First Search (DFS)**

BREADTh FIRST SEARCH (BFS)

- **Breadth-First Search (BFS)** is a **level-order traversal** algorithm for graphs (and trees).

- It starts from a **source vertex** and explores all its **neighbors** (vertices directly connected by an edge) before moving on to their neighbors. In other words, BFS explores the graph **layer by layer**.

- Graphs can have cycles that lead back to already visited nodes. To prevent visiting the same node again, we use a **boolean array** to mark each node once it is processed.

•To manage traversal order, we use a **queue** that follows the **First In First Out (FIFO)** method. This ensures nodes are visited in the same order they were discovered.

BREADTh FIRST SEARCH (BFS) ALGORITHM

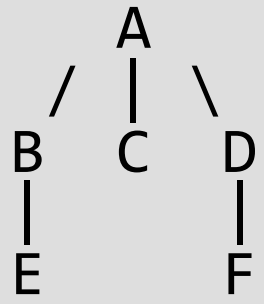
```
BFS(G, start_vertex):
    create an empty queue Q
    mark start_vertex as visited
    enqueue(start_vertex)

    while Q is not empty:
        current = dequeue(Q)
        process(current)
        for each neighbor in
adjacency_list[current]:
            if neighbor is not visited:
                mark neighbor as visited
                enqueue(neighbor)
```

BREADTH FIRST SEARCH (BFS)

EXAMPLE

Consider the following graph:



Adjacency List:

A → [B, C, D]
B → [A, E]
C → [A]
D → [A, F]
E → [B]
F → [D]

Starting from vertex A:

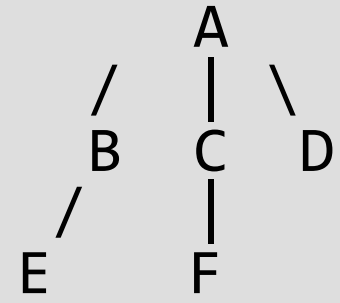
Step	Queue	Visited Nodes	Current Node	Enqueued Neighbors
1	[A]	{A}	-	-
2	[B, C, D]	{A, B, C, D}	A	B, C, D
3	[C, D, E]	{A, B, C, D, E}	B	E
4	[D, E]	{A, B, C, D, E}	C	-
5	[E, F]	{A, B, C, D, E, F}	D	F
6	[F]	{A, B, C, D, E, F}	E	-
7	[]	{A, B, C, D, E, F}	F	-

Traversal Order: A → B → C → D → E → F

BREADTh FIRST SEARCH (BFS)

EXAMPLE

Consider the following graph:



Adjacency List:

A → [B, C, D]
B → [A, E, F]
C → [A, F]
D → [A]
E → [B]
F → [C]

Starting from vertex A:

Step	Queue	Visited Nodes	Current Node	Enqueued Neighbors
1	[A]	{A}	-	-
2	[B, C, D]	{A, B, C, D}	A	B, C, D
3	[C, D, E]	{A, B, C, D, E}	B	E
4	[D, E]	{A, B, C, D, E}	C	-
5	[E, F]	{A, B, C, D, E, F}	D	F
6	[F]	{A, B, C, D, E, F}	E	-
7	[G]	{A, B, C, D, E, F}	F	-

Traversal Order: A → B → C → D → E → F

BREADTH FIRST SEARCH (BFS) APPLICATIONS

- BFS is widely used in many real-world problems:
- **Shortest Path in Unweighted Graphs**
  → BFS finds the minimum number of edges between two nodes.
- **Social Networks**
  → Finding degrees of separation (e.g., "friends of friends").
- **Web Crawlers**
  → Traversing hyperlinks level by level from a seed URL.
- **Network Broadcasting**
  → Sending data packets to all nodes in minimal hops.
- **Finding Connected Components**
  → BFS can help identify all nodes reachable from a given vertex**.**
- **AI and Games**
  → Used for finding the shortest route or state transitions in puzzle-solving.

BREADTH FIRST SEARCH (BFS)

PROPERTIES

1. **Traversal Order**

•BFS explores vertices in **increasing order of their distance** (in edges) from the source node.

•It uses a **queue (FIFO)** data structure to keep track of nodes to visit next.

2. Completeness

- **BFS is complete**, meaning if a solution (or target node) exists, BFS will definitely find it.
- This is true as long as the branching factor (number of neighbors) is finite.

3. **Optimality**

- BFS is **optimal** (i.e., finds the shortest path) **if all edges have equal weight (e.g., weight = 1)**.
- If edges have varying weights, BFS is not guaranteed to find the shortest path — Dijkstra's algorithm is needed instead.

BREADTH FIRST SEARCH (BFS)

PROPERTIES

4. **Time Complexity**

- **O(V+E)**, where
  - V = number of vertices,
  - E = number of edges.
- Each vertex and edge is processed at most once

5. **Space Complexity**

- **O(V)** — since BFS stores all vertices in the queue in the worst case.
- In a dense graph, this can become quite large.

6. **Type of Algorithm**

- It's a **uninformed search algorithm** — it doesn't use any heuristic information about the goal.
- It's also **systematic** — it explores all nodes at a given depth before going deeper.

BREADTH FIRST SEARCH (BFS)

PROPERTIES

7. Use Cases

- Finding the **shortest path** in an unweighted graph.
- **Level-order traversal** in trees.
- Checking **connectivity** or finding **connected components** in a graph.
- **Peer-to-peer networks**, **web crawlers**, **GPS navigation** (when edges are uniform), etc.

8. Properties in Tree Context

When BFS is applied to trees (often called **level-order traversal**):

- Visits nodes level by level (root → children → grandchildren).
- Naturally suited for printing or processing trees by depth.

9. Parent/Predecessor Tracking

- You can maintain a **parent map** (or predecessor array) to reconstruct the path from source to any reachable node.

BREADTH FIRST SEARCH (BFS)

ADVANTAGES

1. Finds the Shortest Path (in Unweighted Graphs)

•BFS guarantees the **shortest path** between the source and any other reachable vertex when all edges have equal weight (e.g., weight = 1).

•This makes it ideal for problems like finding the **minimum number of moves** or **steps** in puzzles and mazes.

2. Completeness

- BFS is **complete** — it will always find a solution if one exists.
- This is because it explores all nodes level by level, ensuring no possible path is missed.

3. Systematic and Simple

- It is **conceptually simple** and easy to implement using a queue.
- Its systematic nature (exploring all nodes at a given depth before going deeper) makes it predictable and reliable.

BREADTH FIRST SEARCH (BFS) ADVANTAGES

4. Useful for Level-by-Level Exploration

- BFS naturally explores nodes by their **distance (in edges)** from thesource.
- This makes it perfect for applications like:
  - **Level-order traversal** of trees.
  - Finding **connected components** in graphs.
  - Determining **shortest routes** in uniform-cost networks.

5. Good for Broadcasting or Spreading Problems

BFS models processes that **expand evenly** from a source, such as:

- **Network broadcasting** (sending a message to all nodes).
- **Social network analysis** (finding degrees of separation).
- **Web crawling** (discovering pages layer by layer).

6. Path Reconstruction Made Easy

- By maintaining a **parent or predecessor map**, BFS allows you to easily reconstruct the **exact path** from the source to any reachable node.

7. Works for Both Directed and Undirected Graphs

- BFS can be applied equally well to **directed** and **undirected** graphs, as long as the graph representation (e.g., adjacency list) is correct.

BREADTH FIRST SEARCH (BFS)

DISADVANTAGES

1. **High Memory Usage**

•BFS stores **all generated nodes in the queue** at each level.

•In a wide or dense graph, this can lead to **exponential memory growth**.

•**Space complexity = O(V)** — which can be problematic for large graphs.

•Example: If each node has a branching factor *b* and the goal is at depth *d*, BFS may store up to **b^d** nodes.

2. **Can Be Slow for Deep or Infinite Graphs**

- BFS explores **level by level**, so if the solution is very deep, it must visit all nodes at shallower levels first.
- For large or infinite search spaces, BFS may take a very long time or never terminate without limits.

3. **Not Suitable for Weighted Graphs**

- BFS assumes all edges have the **same cost (weight = 1)**.
- It **cannot handle variable edge weights**, and will not find the shortest path if edges have different costs.

In that case, **Dijkstra's algorithm** or **A*** should be used instead.

BREADTH FIRST SEARCH (BFS)

DISADVANTAGES

4. **May Explore Unnecessary Nodes**

- BFS explores **all nodes at a given depth**, even if some paths are clearly irrelevant or non-optimal.
- This wastes time and resources when only a small portion of the graph leads to the goal.

5. **Less Efficient for Finding a Single Deep Solution**

- If the solution is deep and unique, **Depth-First Search (DFS)** might reach it faster with less memory.
- BFS, on the other hand, expands many unnecessary nodes before reaching that depth.

6. **Requires Graph to Fit in Memory**

- Since BFS needs to store all nodes (visited and unvisited), it assumes that the entire graph can be **held in memory** — which is not feasible for very large datasets or streaming data.

CONCLUSION

BFS explores all nodes **level by level**, ensuring the shortest path in an unweighted graph.

It uses a **queue** to manage the traversal order.

REFERENCE

- GeeksforGeeks
- W3Schools
- TutorialsPoint
- Wikipedeia

THANK YOU

OUR TEAM

- Aravind Babu [LEAD]
- Hari A Menon
- Chaithanya P
- Anupam Sawanth
- Poornima Jayachandran
- Abhishek S
- Niranjana S Nair